Vision Document

AI-Driven Transparent Repair of Software Dependency Configurations

Author: Muhammad Ibtisam Tanveer
Supervisor: Dr. Sheeba Samuel
Program / University: Msc Web Engineering / TUC
Date: 06, 2026

1. Introduction

Modern software is built using many external libraries, called dependencies. These libraries have multiple versions, and each version expects particular versions of other libraries and of the programming language itself. Together these requirements form the software's dependency configuration. When that configuration is missing, incomplete, or out of date, the software fails to run even when its own source code is correct.

This is especially damaging for shared scientific code such as Jupyter notebooks released alongside research papers. As libraries are upgraded, older versions become unavailable or incompatible, so a notebook that ran when it was published may no longer execute years later. The result is a loss of computational reproducibility: other researchers can no longer re-run the analysis to confirm the reported results. Empirical studies confirm this is widespread: a large fraction of published notebooks can no longer be reproduced, with broken or under-specified dependencies among the leading causes [5].

2. Motivation

Repairing a broken dependency configuration by hand is slow and demands expertise that many researchers and developers do not have. Recent work has therefore sought to automate this repair, increasingly using large language models, and reports promising success rates. However, these tools share a critical weakness: they output a corrected configuration but do not explain it. The developer is given a set of version numbers and must simply trust them, with no way to see why each choice was made, what alternatives were rejected, or whether the result was actually verified.

This thesis addresses that weakness. Its aim is automated dependency repair that is transparent: every fix is accompanied by a clear, checkable explanation, so that developers can understand, verify, and appropriately trust the result.

3. Background and Related Work

Research relevant to this thesis falls into four areas.

3.1 The reproducibility problem

Studies consistently report that code, especially computational notebooks, frequently fail to execute reproducibly at a later date. Dependency-related problems are among the leading

causes of such failures [5]. The existence and impact of these issues highlight a significant threat to scientific reproducibility and provide a strong motivation for research on automated dependency repair.

3.2 Classical dependency resolution

Before LLMs, dependency resolution was treated as a constraint-satisfaction problem: find a combination of versions that satisfies every requirement. Tools relied on knowledge graphs or databases of package information to infer what a program needs [2].
These methods are precise but rigid as they depend on large, well-maintained metadata, struggle with the explosion of possible version combinations, and have difficulty with conflicts among transitive dependencies. Notably, the constraint-solving literature already recognised that when no valid combination exists, the failure must be explained to the developer clearly [13] an early argument for transparency that later AI tools largely dropped.

3.3 LLM-based repair

Recent approaches use LLMs to find and repair dependencies directly from a project's code and its error messages, often combined with retrieval-augmented generation (RAG) and an iterative loop that proposes a configuration, runs it, reads the resulting errors, and tries again [1]. A related line of work argues that, rather than recreating an old environment, it is more reliable to keep the modern environment fixed and update the code to match it [4]. This area also includes a fast-growing set of agent systems and shared benchmarks for automatically setting up project environments, such as EnvBench [3]. Across this work the dominant objective is the repair success rate; the reasoning behind a fix is typically discarded rather than shown to the user.

3.4 Trust and transparency in automated repair

Another research examines whether developers actually accept automatically generated fixes. The main finding is that correctness alone is not enough. Studies show that developers are more likely to trust and use a fix when they are given additional information, such as explanations of the patch and evidence supporting the change [7][8]. Similar results have been reported in industry, where engineers valued explanations and the ability to accept or reject repair suggestions [9].

The same issue exists with LLM-based coding assistants. Developers often distrust these tools when their behavior is unclear or does not match their expectations [11]. Recent research argues that most coding assistants only show the final output and hide the reasoning behind it, making it difficult for users to understand why a solution was generated. To address this, researchers propose adding explanations that make the tool’s reasoning more transparent [10].

This idea is closely related to calibrated trust, which means relying on automation appropriately neither too much nor too little [12]. However, transparency and explanation features are largely missing from current LLM-based dependency repair tools discussed in Section 2.3.

4. Research Gap

Combining the four areas reveals a clear gap. The reproducibility research shows the problem matters [5]. LLM-based tools can now repair many dependency problems automatically [1][4]. The trust research shows that developers need to understand and verify a fix before they will rely on it [7, 8, 9, 10]. Yet current automated dependency-repair tools optimize almost entirely for success rate and produce opaque fixes: they output a working configuration without explaining why each version was chosen, what alternatives were rejected, where each decision came from, or whether the fix was verified. This makes the results hard to trust, hard to audit, and hard to learn from. This thesis aims to close that gap.

5. Research Questions

To address this gap, this thesis investigates the following questions:

RQ1. How effectively can an LLM-based approach repair broken dependency configurations so that previously non-runnable Python projects execute successfully?

RQ2. How can the repair process be made transparent — automatically producing, for each fix, the reason for the change, the source of that decision (verified package metadata versus model suggestion), the alternatives considered and rejected, and a confidence level?

RQ3. Does adding the transparency layer reduce repair effectiveness or efficiency compared with a repair-only baseline?

RQ4. Do the generated explanations help developers form trust accepting correct fixes and detecting incorrect ones?

6. Proposed Approach

The proposed method extends the iterative LLM repair loop used in recent work [1] so that explanation becomes a first-class output rather than an afterthought. At a high level, the tool would:

1. Take a project that does not run (for example, a notebook or script).
2. Execute it in an isolated environment and capture the resulting error messages.
3. Use an LLM, supported by retrieval of trusted package information, to propose a configuration change together with a short reason for it.
4. Verify the change by checking it against authoritative package metadata and by re-running the project; repeat until it works or a limit is reached.
5. Collect the reasons, sources, rejected alternatives, and verification results from every iteration into a single human-readable transparency report.

The transparency report is the novel element. Building on what developers say they need [7, 9] and on explanation layers for code assistants [10], it would make each repair decision carry:

- **Reason —** the constraint or rule that forced the chosen version.
- **Source / provenance —** whether the decision is backed by official package metadata or is an unverified model suggestion.

- **Alternatives —** which other versions were tried and why they failed.
- **Verification —** the result of actually re-running the project after the fix.
- **Confidence —** a signal of how reliable the change is, flagging low-confidence fixes for human review.

Python package versions and requirements are publicly listed on the package index, so the tool can check a model's claim against a trusted source instead of just trusting the model. This matters because models sometimes say a fix worked when it didn't, so verifying it by actually running the code is essential [10].

7. Evaluation Plan

The method would be evaluated along two complementary dimensions:

- **Effectiveness —** count how many broken projects are repaired successfully, using an existing benchmark so results are comparable with prior work. Candidate benchmarks include EnvBench [3], the challenging single-file dataset used by recent LLM repair work [1], and the notebook collection used in the modernization study [4].
- **Transparency and usefulness —** assess whether the explanations help. This could take the form of a small study in which developers review fixes with and without explanations while trust and error-detection are observed, following the design of trust studies in repair research [8]; or, if a human study is not feasible, an automated proxy that measures whether explanations correctly expose deliberately incorrect fixes.

8. Preliminary Timeline

A rough phasing, to be refined with the supervisor:

- **Phase 1 —** finish the literature review and this vision document; reproduce a few broken examples by hand to understand the problem.
- **Phase 2 —** build a minimal repair loop (run, capture error, propose fix, re-run) on a small set of examples (addresses RQ1).
- **Phase 3 —** add the transparency report with reason, source, alternatives, verification, and confidence (addresses RQ2).
- **Phase 4 —** evaluate on a benchmark and compare with a repair-only baseline (addresses RQ3, and RQ4 if feasible); write up.

9. References

[1] A. Bartlett, C. C. S. Liem, and A. Panichella, "The Last Dependency Crusade: Solving Python Dependency Conflicts with LLMs," arXiv:2501.16191, 2025. (Accepted at the 1st Workshop on Agentic Software Engineering, AgenticSE @ ASE 2025.)

[2] H. Ye, W. Chen, W. Dou, G. Wu, and J. Wei, "Knowledge-Based Environment Dependency Inference for Python Programs," in *Proc. 44th Int. Conf. on Software Engineering (ICSE)*, 2022, pp. 1245–1256. doi: 10.1145/3510003.3510127.

[3] A. Eliseeva, A. Kovrigin, I. Kholkin, E. Bogomolov, and Y. Zharov, "EnvBench: A Benchmark for Automated Environment Setup," in *3rd Workshop on Deep Learning for Code (DL4C @ ICLR)*, 2025. arXiv:2503.14443.

[4] B. Jin, K. Wang, and P. Nie, "Automated Modernization of Machine Learning Engineering Notebooks for Reproducibility," arXiv:2602.07195, 2026.

[5] S. Samuel and D. Mietchen, "Computational Reproducibility of Jupyter Notebooks from Biomedical Publications," *GigaScience*, vol. 13, art. giad113, 2024. doi: 10.1093/gigascience/giad113.

[6] Y. Wang, M. Wen, Y. Liu, Y. Wang, Z. Li, C. Wang, H. Yu, S.-C. Cheung, C. Xu, and Z. Zhu, "Watchman: Monitoring Dependency Conflicts for Python Library Ecosystem," in *Proc. 42nd Int. Conf. on Software Engineering (ICSE)*, 2020, pp. 125–135. doi: 10.1145/3377811.3380426.

[7] Y. Noller, R. Shariffdeen, X. Gao, and A. Roychoudhury, "Trust Enhancement Issues in Program Repair," in *Proc. 44th Int. Conf. on Software Engineering (ICSE)*, 2022, pp. 2228–2240. doi: 10.1145/3510003.3510040. arXiv:2108.13064.

[8] T. J. Ryan, G. M. Alarcon, C. Walter, R. Gamble, S. A. Jessup, A. Capiola, and M. D. Pfahler, "Trust in Automated Software Repair: The Effects of Repair Source, Transparency, and Programmer Experience on Perceived Trustworthiness and Trust," in *HCI for Cybersecurity, Privacy and Trust (HCI-CPT 2019)*, Lecture Notes in Computer Science, vol. 11594, Springer, 2019, pp. 452–470. doi: 10.1007/978-3-030-22351-9_31.

[9] D. Williams, J. Callan, S. Kirbas, S. Mechtaev, J. Petke, T. Prideaux-Ghee, and F. Sarro, "User-Centric Deployment of Automated Program Repair at Bloomberg," in *Proc. 46th Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2024, pp. 81–91. doi: 10.1145/3639477.3639756. arXiv:2311.10516.

[10] R. Ye, Z. Zhang, B. Almazroua, and M. Liut, "Beyond Autocomplete: Designing CopilotLens Towards Transparent and Explainable AI Coding Agents," arXiv:2506.20062, 2025. (XLLM-Reason-Plan Workshop @ COLM 2025.)

[11] R. Wang, R. Cheng, D. Ford, and T. Zimmermann, "Investigating and Designing for Trust in AI-Powered Code Generation Tools," in *Proc. 2024 ACM Conf. on Fairness, Accountability, and Transparency (FAccT)*, 2024, pp. 1475–1493. doi: 10.1145/3630106.3658984. arXiv:2305.11248.

[12] J. D. Lee and K. A. See, "Trust in Automation: Designing for Appropriate Reliance," *Human Factors*, vol. 46, no. 1, pp. 50–80, 2004. doi: 10.1518/hfes.46.1.50_30392.

[13] R. Cox, "Version SAT," *research.swtch.com*, Dec. 13, 2016. [Online]. Available: https://research.swtch.com/version-sat